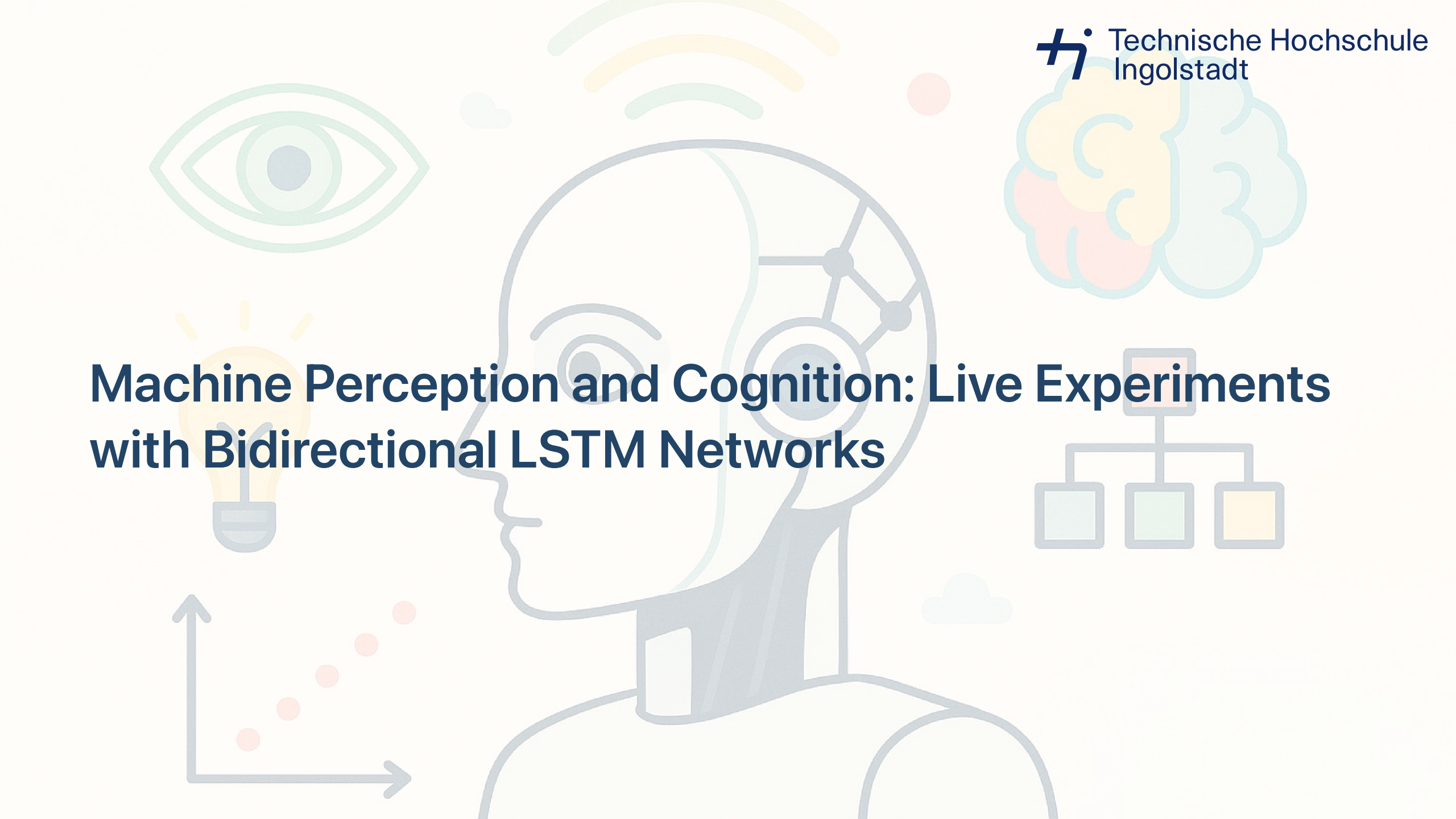
Technische Hochschule Ingolstadt



Machine Perception and Cognition: Live Experiments with Bidirectional LSTM Networks

Live Experiment: Overview

Goal: Train a multi-layer **bidirectional LSTM** to normalize date strings

Input: Human-readable date in one of 10 format variants

- `"15th January 2026"`, `"Jan 15, 2026"`, `"15/01/2026"`, …

Output: ISO 8601 date string

- `"2026-01-15"`

Live Experiment: Overview

Key ideas in this experiment:

- Character-level sequence-to-sequence modeling
- Bidirectional LSTM for full-context processing
- PyTorch `Dataset`, `DataLoader`, `nn.LSTM`, `nn.Embedding`
- Exact-match accuracy and edit distance as evaluation metrics
- TensorBoard for tracking training progress

Task: Date Normalization

Why is this a good demo task?

- **Bidirectionality matters:** `"January"` must be fully read before predicting the month digit `"01"` — right context is essential
- **Fixed, short output length:** ISO 8601 is always exactly 10 characters (`YYYY-MM-DD`)
- **Mixed input styles:** month names, digits, punctuation, and ordinals require a genuine character-level mapping rather than memorizing a single template
- **Easy to measure:** exact-match accuracy and character-level edit distance

Task: Date Normalization

Example pairs:

Input (source)	Target
`"15th January 2026"`	`"2026-01-15"`
`"Jan 15, 2026"`	`"2026-01-15"`
`"15/01/2026"`	`"2026-01-15"`
`"2026/01/15"`	`"2026-01-15"`

Input Format Variants

The model must handle **10 different human-readable date formats**:

Format ID	Example	Pattern
0	`"15th January 2026"`	ordinal + full month name
1	`"January 15, 2026"`	full month name, comma
2	`"Jan 15, 2026"`	abbreviated month, comma

Input Format Variants

Format ID	Example	Pattern
3	`"15/01/2026"`	DD/MM/YYYY
4	`"01/15/2026"`	MM/DD/YYYY

Formats 3 and 4 are **ambiguous** when both day and month are ≤ 12.

For example, `"01/02/2026"` could mean **1 February 2026** (`DD/MM/YYYY`) or **2 January 2026** (`MM/DD/YYYY`).

These cases are **not disambiguable from the string alone** and therefore introduce label ambiguity.

Input Format Variants

Format ID	Example	Pattern
5	`"15-01-2026"`	DD-MM-YYYY
6	`"2026/01/15"`	YYYY/MM/DD
7	`"15 January 2026"`	full month name, no comma
8	`"January 15th, 2026"`	full month name + ordinal
9	`"15.01.2026"`	DD.MM.YYYY

Fixed-Length Sequence Transduction

In this experiment, the model maps a padded input sequence $\mathbf{x} = (x_1, \ldots, x_n)$ to a padded output sequence $\hat{\mathbf{y}} = (\hat{y}_1, \ldots, \hat{y}_n)$ with the **same fixed length** $n = 24$ (`SEQ_LEN = 24`):

- This is a **fixed-length, position-wise transduction** setup: one output distribution is produced at each time step
- Only the first 10 output positions correspond to the ISO 8601 target; the remaining positions are `<PAD>` and are ignored in the loss

Fixed-Length Sequence Transduction

Architecture in this experiment:

$$\mathbf{x}_t \xrightarrow{\text{Embedding}} \mathbf{e}_t \xrightarrow{\text{Bidir. LSTM}} \mathbf{h}_t \xrightarrow{\text{Linear}} \hat{\mathbf{y}}_t$$

No separate encoder/decoder, no attention, no autoregressive decoding loop — this is simpler than a general encoder-decoder seq2seq model with attention, and the entire padded output is predicted in a single forward pass.

Character-Level Representation

The model works directly on **individual characters** — not words or subwords.

Each character is mapped to an integer index:

- `'A'` → `3`, `'J'` → `15`, `'2'` → `1`, `'-'` → `2`, …
- A special `<PAD>` token (index `0`) pads sequences to a fixed length

Why character level?

- The date formats consist of digits, letters, punctuation — a character vocabulary is small (~50 tokens) and complete
- No tokenization ambiguity: every character is treated independently

Character-Level Representation

Fixed sequence length `SEQ_LEN = 24`:

- Source strings are padded to 24 characters
- Target strings (10 chars for `YYYY-MM-DD`) are padded to 24 characters
- The loss ignores the 14 padding positions in the target via `ignore_index`

Vocabulary Class

The `Vocabulary` class builds the character-to-index mapping from all training pairs:

```
class Vocabulary:
    def __init__(self, pairs: list[tuple[str, str]]) -> None:
        chars: set[str] = set()
        for src, tgt in pairs:
            chars.update(src)
            chars.update(tgt)
        self.tokens   = ["<PAD>"] + sorted(chars)   # index 0 is always <PAD>
        self.char2idx = {c: i for i, c in enumerate(self.tokens)}
        self.idx2char = {i: c for i, c in enumerate(self.tokens)}

    def encode(self, s: str) -> list[int]:
        return [self.char2idx[c] for c in s]          # string → list of ints
```

Both source and target strings share the **same vocabulary** — only one `encode` / `decode` method is needed.

PyTorch Dataset

```
class DateSeq2SeqDataset(torch.utils.data.Dataset):
    def __init__(self, pairs, vocab):
        pad = vocab.pad_idx
        self.data = []
        for src, tgt in pairs:
            # Encode source and pad to SEQ_LEN
            src_ids = vocab.encode(src) + [pad] * (SEQ_LEN - len(src))
            # Encode target and pad to SEQ_LEN (only first 10 chars matter)
            tgt_ids = vocab.encode(tgt) + [pad] * (SEQ_LEN - len(tgt))
            self.data.append((src_ids, tgt_ids))
```

PyTorch DataLoader

`torch.utils.data.DataLoader` handles **batching, shuffling, and collation**:

```
def _collate_fn(batch):
    srcs, tgts = zip(*batch)
    # Stack pre-padded sequences into tensors, shape: (SEQ_LEN, batch_size)
    src_tensor = torch.tensor(srcs, dtype=torch.long).t()
    tgt_tensor = torch.tensor(tgts, dtype=torch.long).t()
    return src_tensor, tgt_tensor

loader = torch.utils.data.DataLoader(
    dataset,
    batch_size=128,
    shuffle=True,
    collate_fn=_collate_fn,   # custom stacking function
    num_workers=0,
)
```



PyTorch DataLoader

Time-first tensor layout `(SEQ_LEN, batch)` is the default convention for `nn.LSTM` in PyTorch (avoids an extra transpose inside the model).

Image source: https://www.pexels.com/photo/glass-hourglass-at-sunset-with-sand-falling-37113102/

Embedding Layer: `nn.Embedding`

An **embedding layer** maps integer token indices to dense vectors:

```
self.embedding = nn.Embedding(
    vocab_size,           # number of distinct tokens
    embed_dim,            # output vector dimension (= 64)
    padding_idx=pad_idx, # gradient is zeroed for this index
)
```

What it does:

- Maintains a learnable weight matrix of shape `(vocab_size, embed_dim)`
- For each input index, it looks up the corresponding row
- **`padding_idx`** keeps the `<PAD>` embedding fixed at zero and excludes it from weight updates

Embedding Layer: `nn.Embedding`

Input/output shapes:

Tensor	Shape
Input `src`	`(seq_len, batch)` — long integer indices
Output `embedded`	`(seq_len, batch, embed_dim)` — dense float vectors

`nn.LSTM` in PyTorch

```
self.lstm = nn.LSTM(
    input_size=embed_dim,     # dimension of each input vector
    hidden_size=hidden_dim,   # LSTM hidden state size per direction
    num_layers=n_layers,      # stacked layers
    bidirectional=True,       # run forward + backward LSTM
    dropout=dropout,          # inter-layer dropout (only if num_layers > 1)
)
```

`nn.LSTM` in PyTorch

Input/output shapes (`bidirectional=True`):

Tensor	Shape	Description
`embedded` (input to `nn.LSTM`)	`(seq_len, batch, embed_dim)`	embedded source sequence
`outputs`	`(seq_len, batch, 2·hidden_dim)`	concatenated fwd + bwd hidden states
`h_n`	`(2·num_layers, batch, hidden_dim)`	final hidden state (all layers, both directions)

Bidirectional LSTM

With `bidirectional=True`, **two independent LSTMs** run in parallel:

$$\overrightarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\mathrm{fwd}}(\mathbf{e}_t,\ \overrightarrow{\mathbf{h}}_{t-1}) \quad \text{(left-to-right)}$$

$$\overleftarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\mathrm{bwd}}(\mathbf{e}_t,\ \overleftarrow{\mathbf{h}}_{t+1}) \quad \text{(right-to-left)}$$

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t\ ;\ \overleftarrow{\mathbf{h}}_t] \quad \text{(concatenated, size } 2 \cdot \text{hidden_dim)}$$

Why bidirectionality matters for this task:

The month name `"January"` appears at the start of format 1 (`"January 15, 2026"`) but at positions 4–10 of format 0 (`"15th January 2026"`). In both cases, the model must have **read the entire month name before committing to the digit `"01"`**. The backward LSTM provides right context at every position — this is impossible with a unidirectional LSTM.

Stacked LSTM Layers

With `num_layers=2`, the output of layer 1 becomes the input of layer 2:

$$\text{Layer 1: } \mathbf{e}_t \to \mathbf{h}_t^{(1)} \in \mathbb{R}^{2 \cdot \text{hidden_dim}}$$

$$\text{Layer 2: } \mathbf{h}_t^{(1)} \to \mathbf{h}_t^{(2)} \in \mathbb{R}^{2 \cdot \text{hidden_dim}}$$

Benefits of stacking:

- Each layer builds **progressively more abstract** representations
- Layer 1 learns low-level character patterns; layer 2 learns higher-level structure (month names, digit groups)

Model Architecture

```
class BidirectionalLSTMSeq2Seq(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim,
                 n_layers, pad_idx, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
                                      padding_idx=pad_idx)
        self.lstm = nn.LSTM(embed_dim, hidden_dim,
                            num_layers=n_layers,
                            bidirectional=True,
                            dropout=dropout if n_layers > 1 else 0.0)
        self.dropout = nn.Dropout(dropout)
        self.fc_out  = nn.Linear(2 * hidden_dim, vocab_size)
```

Forward Pass

```
def forward(self, src: torch.Tensor) -> torch.Tensor:
    # src: (seq_len, batch)  -  integer character indices

    # 1. Look up dense embeddings and apply dropout
    embedded = self.dropout(self.embedding(src))
    # embedded: (seq_len, batch, embed_dim)

    # 2. Run bidirectional LSTM over the full sequence
    outputs, _ = self.lstm(embedded)
    # outputs: (seq_len, batch, 2*hidden_dim)
    # _      : (h_n, c_n) - final states, not needed here

    # 3. Project each position's hidden state to vocabulary logits
    logits = self.fc_out(outputs)
    # logits: (seq_len, batch, vocab_size)

    return logits
```

Training Loop

```
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss(ignore_index=pad_idx)

for epoch in range(N_EPOCHS):
    model.train()
    for src, tgt in dataloader:
        optimizer.zero_grad()          # 1. clear old gradients

        logits = model(src)            # 2. forward pass
        # logits: (seq_len, batch, vocab_size)
        # tgt:    (seq_len, batch)

        out_flat = logits.reshape(-1, logits.size(2))  # (seq_len*batch, vocab_size)
        tgt_flat = tgt.reshape(-1)                     # (seq_len*batch,)

        loss = criterion(out_flat, tgt_flat)           # 3. compute loss
        loss.backward()                                # 4. backpropagate

        optimizer.step()               # 5. update weights
```

Cross-Entropy Loss with `ignore_index`

```
criterion = nn.CrossEntropyLoss(ignore_index=pad_idx)
```

What `ignore_index` does:

- Positions in the target tensor where the label equals `pad_idx` are **excluded from the loss and its gradient**
- Target layout: `[y_0, y_1, …, y_9, PAD, PAD, …, PAD]` (10 real chars + 14 padding)
- Only the first 10 output positions receive a gradient signal

Cross-Entropy Loss with `ignore_index`

Why reshape before computing the loss?

`nn.CrossEntropyLoss` expects:

- Predictions: `(N, C)` where `N` = number of samples, `C` = number of classes
- Targets: `(N,)` integer labels

The reshape merges the time and batch dimensions: `N = seq_len × batch_size`.

Switching Between Training and Evaluation Mode

```
# Training
model.train()     # enables dropout
for src, tgt in dataloader:
    ...

# Evaluation / inference
model.eval()                          # disables dropout → deterministic predictions
with torch.no_grad():                 # skips autograd graph → saves memory and time
    logits = model(src)
    pred_ids = logits.argmax(dim=2)
```

`model.eval()` sets all `nn.Dropout` layers to pass-through mode (no random zeroing).

`torch.no_grad()` tells PyTorch not to track gradient computations — there is no backward pass during inference, so no computation graph is needed. This reduces memory usage significantly for large models.

Evaluation: Exact-Match Accuracy

```
def evaluate_accuracy(model, pairs, vocab, device):
    n_correct = 0
    for src, tgt in pairs:
        pred = translate(model, src, vocab, device)
        if pred == tgt:                # full string must match exactly
            n_correct += 1
    return n_correct / len(pairs)  # fraction in [0, 1]
```

Strict binary criterion: the predicted 10-character ISO 8601 string must equal the ground-truth string character-by-character. A single wrong digit counts as failure.

Limitation: a prediction of `"2026-01-16"` for target `"2026-01-15"` scores 0, even though 9 out of 10 characters are correct. This motivates a complementary metric.

Evaluation: Edit Distance

Levenshtein edit distance counts the minimum number of single-character **insertions, deletions, or substitutions** to transform the prediction into the target:

Prediction	Target	Edit Distance
`"2026-01-15"`	`"2026-01-15"`	0 (perfect)
`"2026-01-16"`	`"2026-01-15"`	1 (one substitution)
`"2026-01-5"`	`"2026-01-15"`	1 (one insertion)
`"2027-03-09"`	`"2026-01-15"`	5

Mean edit distance over the test set shows how many characters the model is getting wrong on average, even when exact-match fails

Inference: The `translate` Function

```
def translate(model, src_str, vocab, device) -> str:
    pad     = vocab.pad_idx
    # 1. Encode and pad the source string to SEQ_LEN
    src_ids = vocab.encode(src_str) + [pad] * (SEQ_LEN - len(src_str))
    # 2. Create a (SEQ_LEN, 1) tensor — batch size = 1
    src     = torch.tensor(src_ids, dtype=torch.long).unsqueeze(1).to(device)

    model.eval()
    with torch.no_grad():
        logits   = model(src)                                  # (SEQ_LEN, 1, vocab_size)
        pred_ids = logits.argmax(dim=2).squeeze(1).tolist()  # list of SEQ_LEN ints

    # 3. Decode only the first ISO_DATE_LEN = 10 predictions
    return vocab.decode(pred_ids[:ISO_DATE_LEN])
```

TensorBoard Logging

```
from torch.utils.tensorboard import SummaryWriter

writer = SummaryWriter()   # creates a 'runs/...' directory

# Inside the training loop, after every epoch:
writer.add_scalar("Loss/train",                avg_loss, epoch)
writer.add_scalar("Accuracy/test_exact_match", acc,      epoch)
writer.add_scalar("EditDistance/test_avg",     avg_ed,   epoch)

writer.close()
```

Visualize training progress:

```
tensorboard --logdir=runs
```

Open `http://localhost:6006` in a browser.

Recap!

Task and data:

- Date normalization: 10 human-readable formats → ISO 8601 (`YYYY-MM-DD`)
- Character-level tokenization; shared vocabulary for source and target; padding to `SEQ_LEN = 24`

PyTorch building blocks:

- `nn.Embedding` — maps integer indices to dense vectors
- `nn.LSTM` with `bidirectional=True` and `num_layers=2` — processes the full sequence in both directions
- `nn.CrossEntropyLoss(ignore_index=pad_idx)` — ignores padding positions
- `optim.Adam` — updates the trainable parameters from the computed gradients

Recap!

Training and evaluation:

- `model.train()` / `model.eval()` toggles dropout
- `torch.no_grad()` skips autograd during inference
- Exact-match accuracy: strict binary criterion
- Mean edit distance: measures how many characters are wrong on average

Recap!

Live experiment:

- Run `python example_lstm_seq2seq.py`
- Watch training loss and accuracy with `tensorboard --logdir=runs`
- Inspect sample predictions printed after training
- Experiment: change `N_LAYERS`, `HIDDEN_DIM`, or `bidirectional` and observe the effect on convergence